

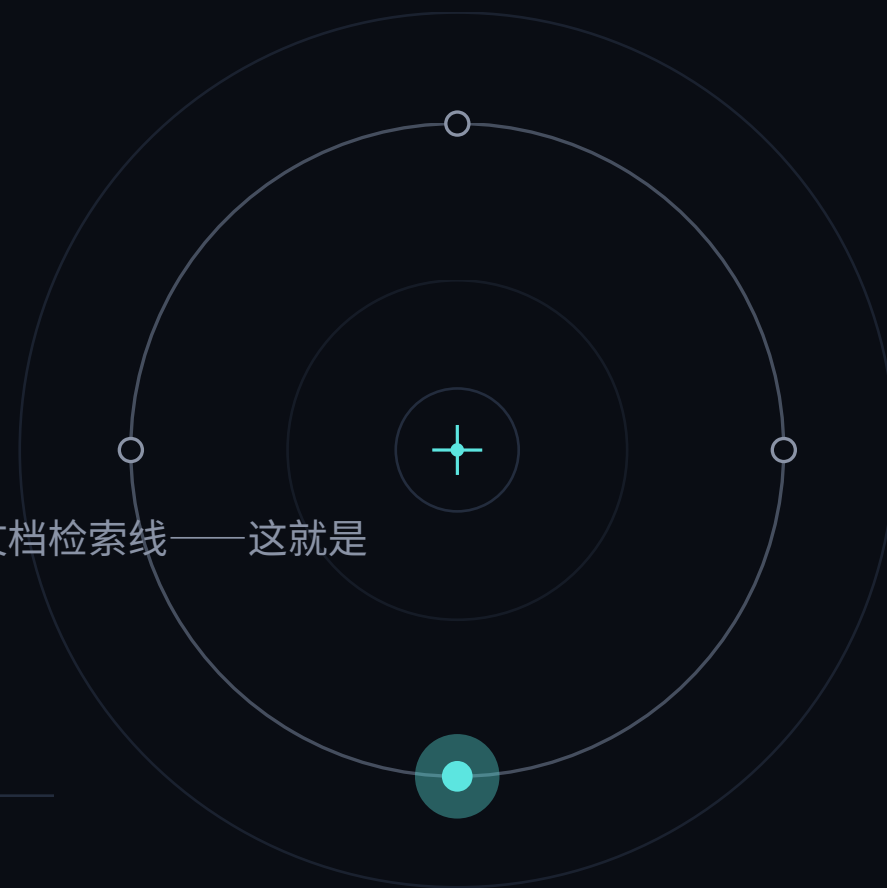
AI AGENT 项目实战营 · 第 08 节

检索工程与向量数据库

先把候选找全，再把真正相关的排上来

本节是讲授 + 实操（60 分钟）。用 50 条真实岗位描述当语料，亲手搭一条文档检索线——这就是 RAG 的地基。

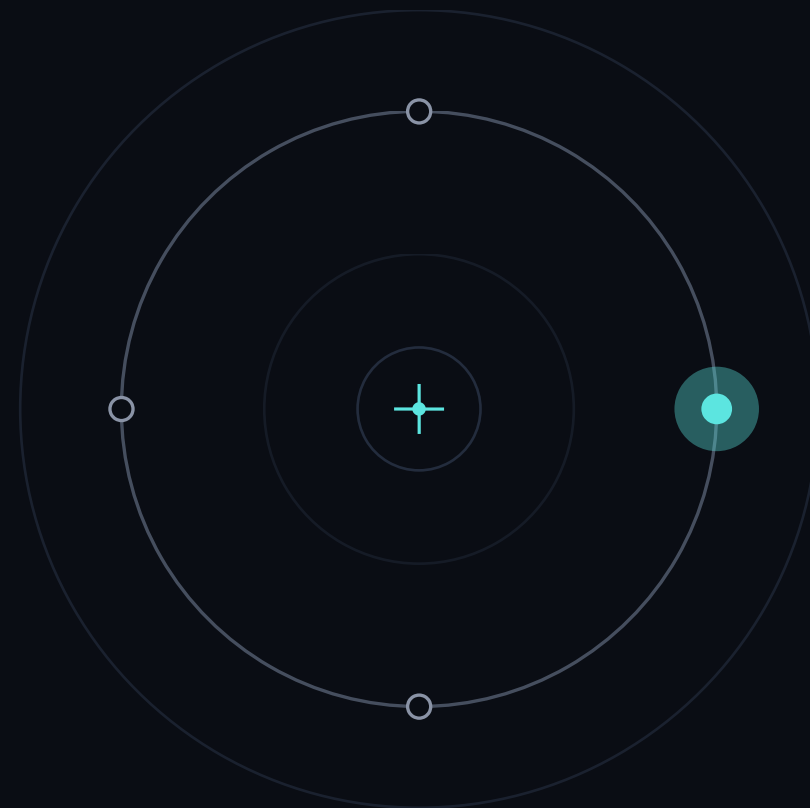
PROJECT · ResuMatch · LECTURE + LAB · 60 MIN



00

PART 0 · OPENING

从这节课你能造什么



问它一句，它翻资料回答

你问 『有适合我的数据分析岗吗？』

① 收 50 条 JD

真实岗位描述



② 变成向量

一句一串数



③ 按意思召回

最像的 3 条



④ 拼进回答

只讲召回到的

OUTPUT 它没把 50 条全塞给模型，只找回最相关的 3 条 JD，据此作答并注明是哪几条。这就是 RAG（先检索、再作答）的样子。

关键词搜 vs 按意思搜

关键词搜 · 对不上就漏

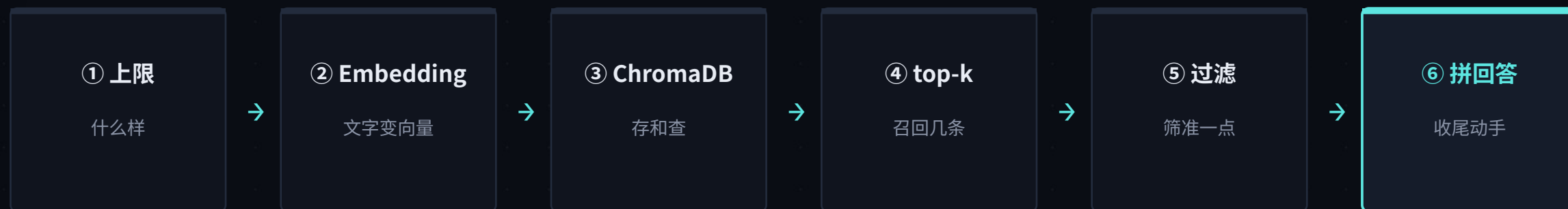
- 查『数据分析』只匹配这四个字
- JD 写 data analytics 就漏了
- 换个说法就查不到
- 像老式站内搜索框

按意思搜 · 换说法也能中

- 先把文字变成『意思的坐标』
- 意思接近，坐标就接近
- 中英、同义、改写都能召回
- 这就是语义检索

语义检索 = 不靠字面一样，靠『意思』的远近找回——今天就搭这条线。

今天走六步

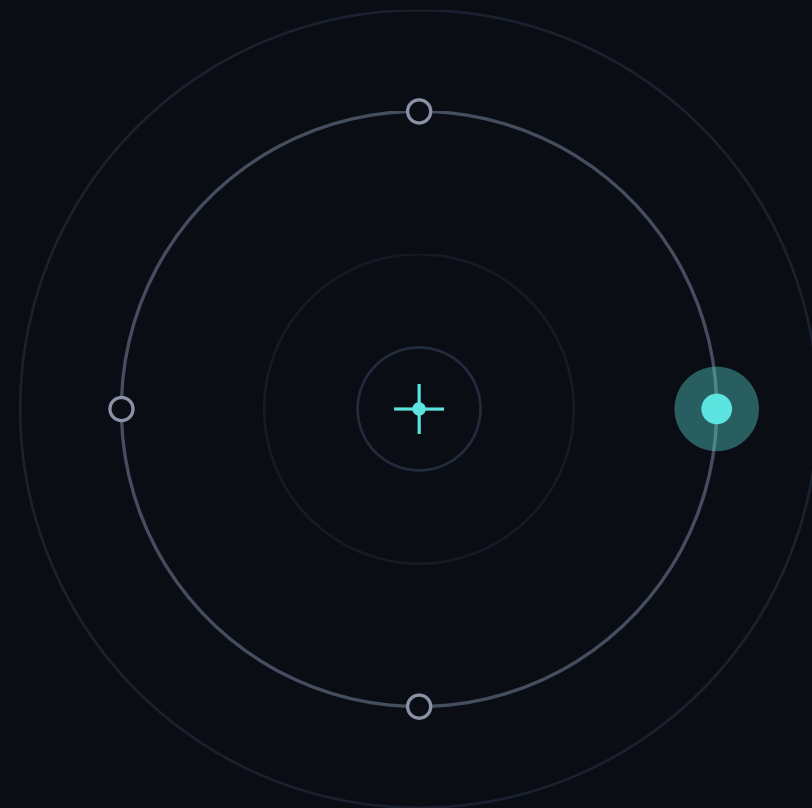


每一步只加一个新东西，且都在 Cursor 里跑给你看。第 ⑥ 步是今天的落点。

01

PART 1 · WHAT GOOD LOOKS LIKE

上限长什么样



四条达标线



做真事、能验证

真存进向量库、真检索得回，不是嘴上说



处理真实数据

语料是 50 条真实 JD，不是三五句造的



不编造

答案只讲召回到的资料，说不清就说没有



会失败还不崩

检索为空、后端缺失，能降级不甩报错

这四条是每个项目的底线，今天的检索线就照它验收。

三种项目都成立

RESUMATCH

求职助手

把 50 条真实 JD 收进向量库，
按意思召回和你最贴的岗位。

COURSE RAG

课程问答助手

检索课程材料，
答案以召回到的内容为准、
注明出处——材料外不编。

SCHEDULE

个人日程助手

检索历史日程 / 笔记，
按意思找回相关的几条，
再据此安排。

套路一样：把资料收进向量库，用问题按意思召回几条，据此作答。今天 demo 走 ResuMatch。

先检索，再作答

一句话：RAG = 回答前先去资料库找几条相关的，只照这几条说话。

用户问题
『有数据岗吗？』



检索资料库
召回最像的 3 条 JD



据这几条作答
不许自由发挥

WHY 资料太多塞不进模型，也不能让它凭记忆瞎编。先检索缩到几条、再作答——省钱、更准、可追溯。

50 条真实 JD 当资料

语料长这样（每条一段岗位描述）

jd_007 数据分析师 · SQL / Python

jd_012 后端工程师 · Go / 微服务

jd_023 产品经理 · B 端 / 数据驱动

jd_031 算法工程师 · 推荐系统

... 共 50 条，各带 role / seniority 标签

为什么用真 JD

真实数据——达标线第 ② 条，不是造几句糊弄

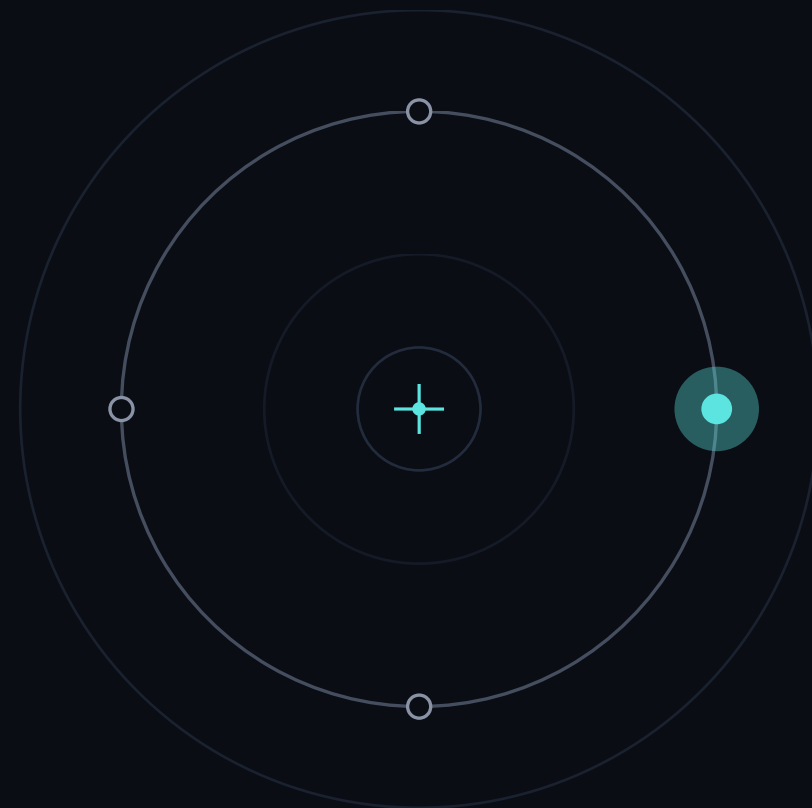
有噪声——正好练『怎么把不相关的筛掉』

能验证——召回对不对，你自己一眼看得出

02

PART 2 · EMBEDDING

文字变向量



Embedding 是什么

一句话：Embedding = 把一段文字压成一串数，意思接近的文字，数也接近。

文字
『数据分析师』



Embedding 模型
nomic-embed-text



向量
[0.12, -0.34, ...] 768 维

WHY 变成向量后，『意思像不像』就成了『坐标近不近』——能用数学算、能排序、能检索。这就是语义检索的地基。

算两两相似度（demo 01）

```
from memory_kit import embed, cosine
```

```
q = embed(["data analyst"])[0]
```

```
jd = embed(["数据分析师岗位"])[0]
```

```
# 余弦相似度：越接近 1 越像
```

```
print(cosine(q, jd))
```

```
# 中英不同字，意思一样 → 分很高
```

OUTPUT

+0.81 数据分析师岗位

+0.66 负责 SQL 取数与报表

+0.09 资深后端 Go 工程师

-0.03 这台电脑续航很长

*查询是英文，命中的是中文——
字不一样，意思一样，分照样高。*

为什么英文也能命中中文？

一句话：Embedding 不比『字』，而是把句子放进一张『意思地图』。

data analyst

字不一样

『数据分析师』



意思一样

都在说：分析数据的职业



位置很近

所以相似度很高

WHY 所以比较的不是中英文字，而是两个点在『意思地图』上离得近不近。

模型怎样学会这张地图？

同一个意思：拉近

- data analyst
- 数据分析师
- 负责分析数据的岗位 → 应该靠近

不同的意思：推远

- data analyst
- 资深 Go 后端工程师
- 这台电脑续航很长 → 应该远离

训练 做错一次，模型就调整一点；练习很多次后，它就学会了这张『意思地图』。

相似度 vs 距离

cosine 相似度

- 越大越像，1 = 最像
- 0 = 没关系，负 = 相反
- 自己算相似度时用它

distance 距离

- 越小越像，0 = 最像
- 换算：distance $\approx$ 1 - cosine
- ChromaDB 的 query 返回它

照做 看到 ChromaDB 返回『分越小排越前』别意外——它是 distance。把这俩搞反，top-k 就全反了。

本地跑真 embedding



设了 Key

用 OpenAI text-embedding-3-small，生产级效果



没 Key 有 Ollama

自动用本地 nomic-embed-text，真向量、零成本、可断网



两者都没

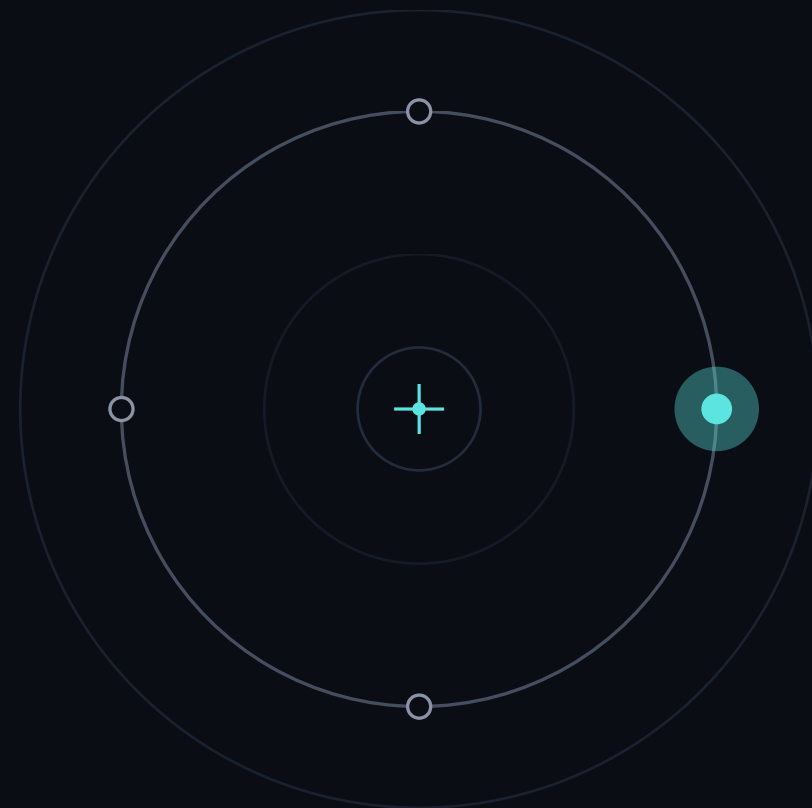
友好提示怎么装，不甩报错

这节不再用『假向量』——离线也走真 embedding，召回质量和上课看到的一致。

03

PART 3 · CHROMADB

向量库：存和查



collection • add • query

COLLECTION

一张表

存一类资料，按名字拿：
`get_or_create_collection`

ADD

写进去

给 `ids + documents`，
自动编码成向量存好。

QUERY

查回来

给一句话 + `n_results`，
返回最像的 `top-k`。

PERSIST `PersistentClient(path="./chroma_db")` —— 落盘到本地目录，重启不丢；忘了设 `path` 就重启即丢。

建库、灌 50 条 JD（demo 02）

```
import chromadb; from jobs import JDS

client = chromadb.PersistentClient(
    path="./chroma_db")
col = client.get_or_create_collection("jobs")

# 50 条 JD 一次灌进去（自动编码）
col.add(ids=ids, documents=texts,
        metadatas=metas)
```

OUTPUT

库内现有 50 条 JD

每条带 metadata：

```
{role:"analyst",
  seniority:"junior"}
```

再跑一次 *count* 不翻倍——
相同 *id* 自动去重，数据也还在。

query 召回 top-k (demo 02)

```
r = col.query(  
    query_texts=[" 数据分析相关 "],  
    n_results=3)  
  
for doc, dist in zip(  
    r["documents"][0], r["distances"][0]):  
    print(round(dist,2), doc)
```

OUTPUT distances

0.31 数据分析师 · SQL

0.44 BI 分析 · 报表

0.52 增长分析 · 数据驱动

问的是『数据分析相关』，
三条都沾边—— distance 小的排前。

检索线的完整链路

平时把资料灌进去，提问时按意思查回来——两排同一套动作。

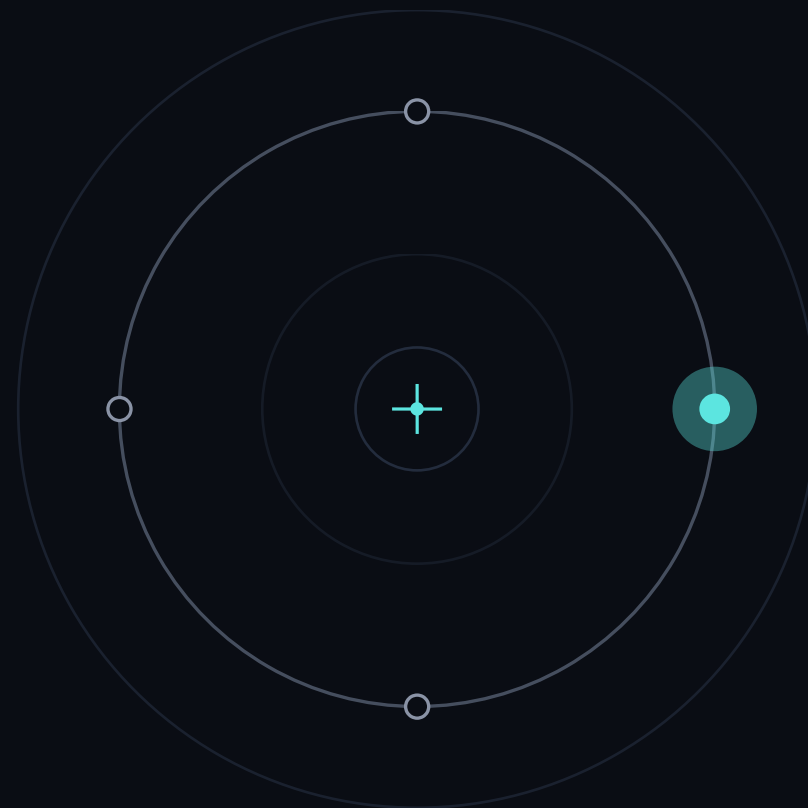


THE KEY 平时『灌』，提问时『查回来拼进去』。这条链就是今天 demo 01→05 串起来的全部。

04

PART 4 · RETRIEVE RIGHT

把检索做对



召回几条最合适

k 太小 · 漏

- 一只召回 1 条
- 一稍微换个说法就漏掉
- 一相关的没进来

k 太大 · 噪声

- 一召回 20 条
- 一大半不相关
- 一把有用的挤下去、还费 token

照做 入门先设 $k=3 \sim 5$ ，再看召回结果调。k 是『要几条』，不是越多越好。

按标签先筛一刀（demo 03）

光靠意思还不够——先用 metadata 把范围缩小，再在里面按意思排。

```
# 只在『初级』岗位里按意思找
r = col.query(
    query_texts=["data analyst"],
    n_results=3,
    where={"seniority": "junior"}
)
```

where= 先筛，再排

先按标签把范围缩到
『初级岗』，
再在里面按意思排序。

资深岗根本不进候选——
又快又准。

不够像的就丢掉（demo 04）

top-k 一定会给你 k 条，哪怕全不相关——所以要再卡一道 distance 阈值。

```
THRESHOLD = 0.3    # 距离超过就当不相关
```

```
kept = [  
    doc for doc, dist in hits  
    if dist < THRESHOLD  
]
```

```
# kept 为空 → 老实说『没找到』
```

检索到的才可信

离太远的那几条，
宁可丢掉也别拼进去。

全被丢光就如实说
『资料里没有』——
这就是『不编造』。

检索到的才可信

1

答案只来自召回的资料

模型不许拿训练里的记忆瞎补

2

召回为空就说没有

别硬凑一个像样的答案

3

能追溯到是哪几条

答完标出用了 jd_007 / jd_012

这三条就是课程 RAG 版的『不编造』——答案有据、说不清就说没有。

召回拼进回答（demo 05）

```
# 用本次问题检索、卡阈值后剩下的
kept = retrieve(question, k=3)

prompt = f"""
【可用资料】
{docs}
【问题】 {question}
只根据上面资料回答，没有就说没有。 """
```

PROMPT · 拼好的

【可用资料】

- jd_007 数据分析师…
- jd_019 BI 分析…

【问题】

有适合我的数据岗吗？

模型只看这两条资料作答，
不许自由发挥。

每问必查，还是它自己决定查

固定管道 · 每问必查

- 流程写死：检索 → 拼资料 → 作答
- 每一问都烧一次检索，闲聊也查
- 答案永远有据，最稳
- 适合：纯问答（客服 FAQ / 课程 RAG）

自主检索 · agentic search

- 检索包成一把工具（@function_tool）
- 模型自己决定：查不查、用什么词、查几次
- 闲聊不烧检索，该查才查
- 适合：既聊天又干活的助手（你的选题）

demo 05 就是右边这种 search_jobs 是一把工具，Agent 判断何时查库；纯问答项目照左边写死管道即可。

两种检索一起找，少漏正确资料

向量检索 · 擅长换说法

- 跨语言、同义词、改写能命中
- 对短缩写、编号、专有名词可能不稳
- 输出一组语义候选

BM25 · 擅长精确词

- 缩写、产品名、法规编号一字不差
- 换一种表达就可能漏掉
- 输出一组关键词候选

Hybrid（混合检索） 先把两路结果合在一起。这一步只关心『正确资料有没有被找到』； RRF 是一种可选合并方法，知道名字即可。

候选找全后，再认真排一次



模型重排

- 重排模型同时阅读问题和每条候选
- 通常更准，但要逐条计算，速度更慢
- 所以只对前 20 条左右的小集合使用

业务重排

- 地点、签证、权限、库存做硬过滤或加权
- 规则可解释、能写测试
- demo 07 用这一种

生产排序常是：语义相关性 + 业务约束，而不是只信一个向量距离。

先把一份资料切成能回答问题的块

只按固定字数切

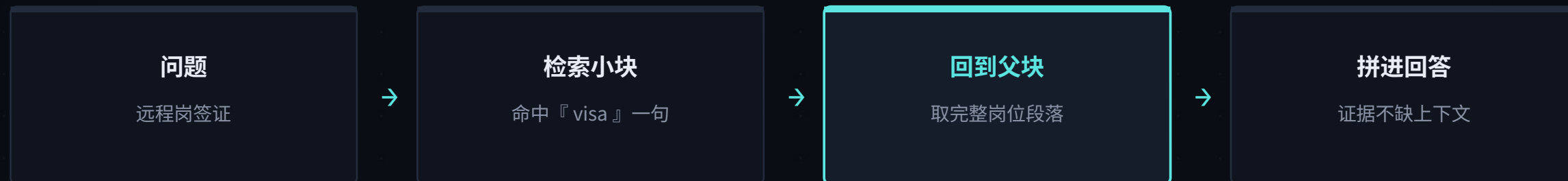
- 实现快，块大小容易控制
- 可能把标题和正文切开
- 『必须条件』留在上一块，签证说明落到下一块
- 检索命中半句话，模型拿不到完整证据

优先按结构切

- 先按标题、段落、列表等自然边界
- 过长段落再按长度二次切
- 每块带 section、source、chunk_id
- 一句证据和它的上下文尽量放在一起

分块目标不是『每块一样长』，而是『用户问到一个点时，召回的块足够独立回答』。demo 08 直接比较两种切法。

小块负责找，大块负责给模型看



块太大

- 一个向量混了太多主题
- 相似度被无关段落稀释
- 召回后还浪费输入 token

块太小

- 容易精确命中一句
- 但证据断掉，模型看不懂前因后果
- parent-child：检索小块，返回父块

这是检索粒度和阅读粒度分开：找的时候要准，给模型时要完整。

先判断问题该怎么搜

原样搜

包含编号、技能名、法规名

保留 AWS、JD-104、H-1B 这类精确词

改写搜

口语很绕、指代很多

把『它那个要求』补成完整查询

拆成两问

一个问题混了两个条件

分别找岗位要求和签证政策，再合并

不检索

寒暄、确认、纯操作命令

别为了走流程每一轮都查库

查询改写不是越聪明越好。精确词一旦被改丢，语义检索和关键词检索都会一起失手。

资料改了，索引也要跟着改



```
chunk_id = sha256(source_id + section)
collection.upsert(ids=ids, documents=chunks)
collection.delete(ids=stale_ids)
```

为什么要稳定 id

- ✓ 同一块重跑不会越存越多
- ✓ 资料更新能覆盖旧版本
- ✓ 源文件删了，能找到对应旧块一起删

只会 add 的索引，跑几周后一定出现重复、过期和删不干净。

权限过滤必须先于相似度排序

错误想法 先全库向量检索，再在返回结果里删掉别人的资料。越权内容已经进入候选，日志和重排阶段都可能看见。

```
collection.query(  
  query_texts=[question],  
  where={"tenant_id": tenant, "acl": role},  
)
```

三条底线

- tenant/user 从登录态取，不信模型生成
- 共享资料和私有资料用明确 ACL
- 写一条 Bob 永远搜不到 Alice 的测试

metadata 过滤既能提准，也能做候选范围控制；真正的授权仍要由应用或权限系统执行。

换 Embedding 模型，不是在原库上改个名字

直接替换的风险

- 新旧向量维度可能不同，查询直接报错
- 维度相同也不代表向量空间相同
- 旧文档用模型 A，新问题用模型 B，距离失去意义
- 阈值也不能照搬旧值

安全迁移

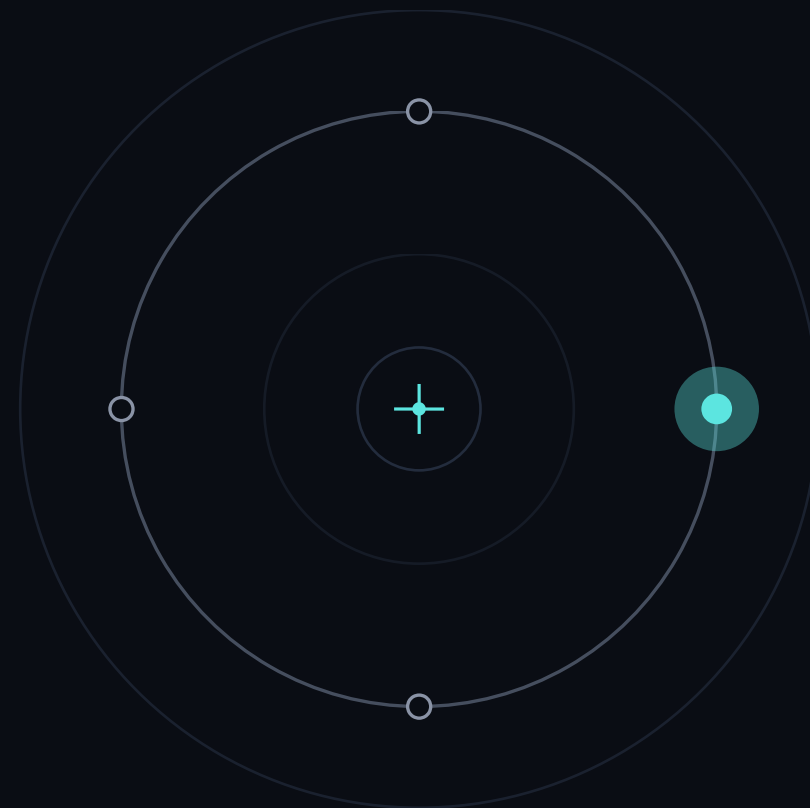
- 新建 collection_v2，完整重算向量
- 写入 embedding_model、dimension、schema_version
- 用固定查询对照 v1/v2 结果
- 一切读流量后保留回滚窗口，再删旧库

向量库里的数值只同一套 embedding 规则里可比较。模型升级 = 数据迁移，不是配置热切换。

05

PART 5 · HANDS-ON

上手与收尾



带检索的 Agent（demo 05）



+ `search_jobs` 工具 @function_tool：把上面检索包成一把工具，模型自己决定何时查库——这就是可调用的 RAG。

★ 学生填空：把语料换成你选题的资料（课程材料 / 你的日程），检索线照搬——这就是你的 RAG 起点。

七个主线 demo + 一个课后小实验

01	02	03	04	05	06	07
Embedding	Chroma	Metadata	Threshold	RAG Agent	Hybrid	Rerank
语义距离	存与查	先过滤	拒绝低相关	带依据回答	两路召回	业务精排

课上按 01→05 跑主线；06/07 看两路召回和业务重排。08-11 把分块、父子块、索引更新和查询规划补齐。

四个离线 demo：检索不是只有 query

08 chunking_intuition.py

固定长度 vs 按标题切

09 parent_child_retrieval.py

小块命中后返回完整父块

10 index_lifecycle.py

upsert 新版并删除旧块

11 query_planner.py

原样搜、改写、拆问或不检索

四份都不需要向量服务。先把数据结构和边界规则跑明白，再替换成真实 embedding。

动手：搭你的检索线

T1 起步

跑通 01-04，看懂『向量→检索→过滤』；把 02 的 query 换成你想问的一句

T2 进阶

把 03 的 where= 和 04 的阈值都用上，让召回既准又不带噪声

T3 挑战

把语料换成你选题的真实资料，接进 05 的 Agent，跑通一次可追溯的 RAG 问答

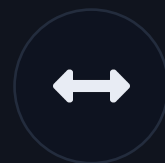
装不上 chromadb 也别卡住——看横幅，local-json 就是兜底生效了，先把闭环跑通。

四个高频坑



存查用了不同 embedding

换了模型向量就对不上——集中成一个常量，存查都从这取



相似度 vs 距离搞反

cosine 大 = 像，distance 小 = 像；query 返回的是 distance



不卡阈值，硬凑答案

top-k 一定给满 k 条——离太远的要丢掉，不然就编造了



忘了设持久化 path

没 PersistentClient(path=...) 就重启即丢，白灌一遍

选题两轨

轨道 A · 跟旗舰 ResuMatch

- 直接照搬今天的检索线
- 把语料换成你收集的岗位 JD
- 上手快、有完整参照

轨道 B · 自选题

- 课程 RAG：检索课程材料，答案以材料为准
- 日程助手：检索历史日程
- 共同点：真资料、可追溯、不编造

不管哪轨，都要过那四条达标线；今天的检索线就是你 RAG 的地基。

作业两档

基础 · 必做

- 用一批真实资料建库 (≥ 30 条)
- 一次 query 召回 top-k 并截图
- 答案只讲召回到的、注明是哪几条

进阶 ★ · 加分

- 加 metadata 过滤 + distance 阈值
- 召回为空时如实说『没有』
- 接进 05 的 Agent，做一次可追溯问答

可追溯 = 答完能说出用了哪几条资料。基础保过线，进阶练的是 RAG 真功夫。

下节：会更新的记忆

① 今天你搭好了检索线

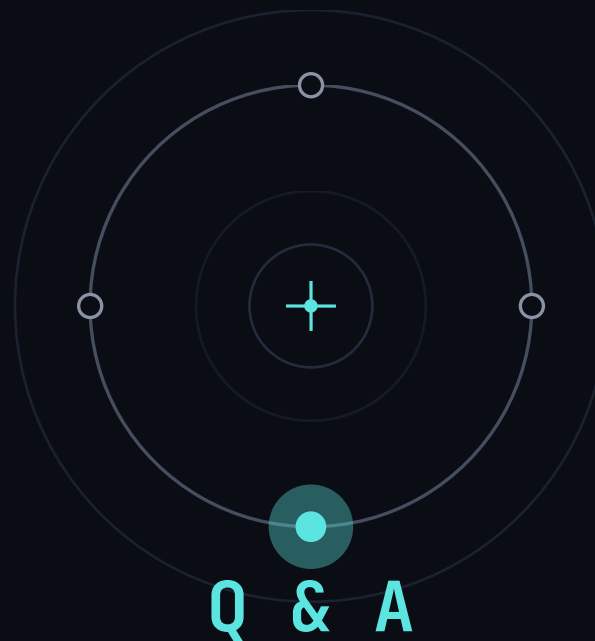
资料灌进向量库、按意思召回、卡准再作答

② 但记忆会变——旧的怎么办

用户说『其实我只有 2 年经验』，之前记的『5 年』就该失效

课前必看 《Demo 09 · 记忆更新与双层协同》18 分钟。这周把检索线搭起来，下节见。

短期 + 长期双层记忆协同，重点讲『记忆更新』怎么不打架



现在它能从一堆资料里，按意思找回该看的那几条了。

跑不通的、装不上 chromadb 的，把报错贴群里——我们一个个解决。

作业：建库 + 召回截图 · 答案只讲召回到的 · 看 Demo 09